

Deep Learning(CAI-466)

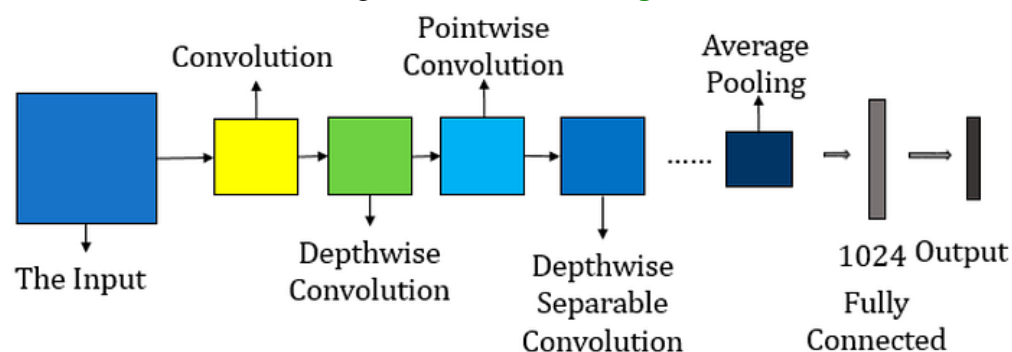
Objectives:

- **MobileNetV1**
 - **Depth-wise separable convolution**
- **MobileNetV2**

MobileNet:

In the realm of computer vision, the demand for lightweight yet powerful models has surged, driven by the need to deploy applications on resource-constrained devices. Enter MobileNet, a groundbreaking architecture that optimizes computational requirements while delivering impressive results. In this article, we will explore the simple MobileNet architecture and compare it with traditional Convolutional Neural Networks (CNNs).

“Using MobileNets will allow you to build and deploy neural networks that work even in low compute environment such as mobile phones” ~ Andrew Ng



MobileNet is a simple but efficient and not very computationally intensive convolutional neural networks for mobile vision applications. MobileNet is widely used in many real-world applications which includes object detection, fine-grained classifications, face attributes, and localization. In this lecture you will learn how exactly it becomes the most efficient and lightweight neural network.

1. Depth-wise separable convolution
 - Depth-wise convolution
 - Point-wise convolution
2. The entire Network Structure
3. Parameters of MobileNet
 - Width-wise Multiplayer
 - Resolution-wise Multiplayer

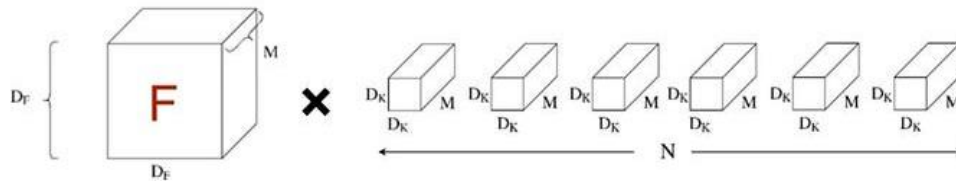
1. Depth-wise separable convolution

The **Depth-wise separable convolution** is comprising of two layers, the depth-wise convolution, and the point-wise convolution. Basically the first layer is used to filter the input channels and the second layer is used to combine them to create a new feature.

Depth-wise convolution

The depth-wise convolutions are used to apply a single filter into each input channel. This is different from a standard convolution in which the filters are applied to all of the input channels.

Let's take a standard convolution,



Standard Convolution

From the above image, the computational cost can be calculated as :

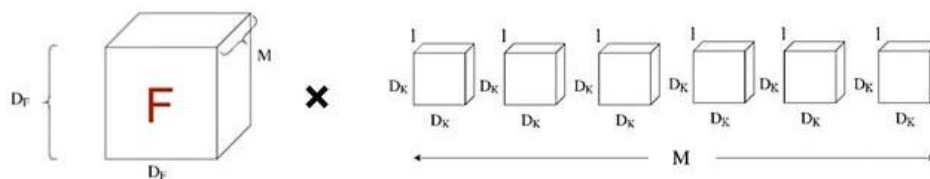
$$D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F$$

Standard Convolution Cost

Where D_F is the spatial dimensions of the input feature map and D_K is the size of the convolution kernel. Here M and N are the number of input and output channels respectively.

For a standard convolution, the computational cost depends multiplicatively on the number of input and output channels and on the spatial dimensions of the input feature map and convolution kernel.

In case of depthwise convolution, as seen in the below image, contains an input **feature map of dimension $D_F \cdot D_F$ and M number of kernels of channel size 1.**



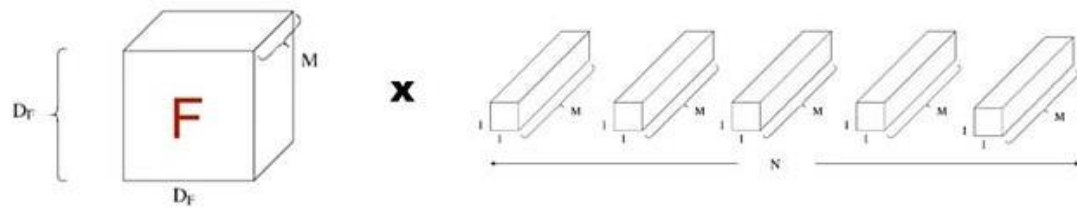
Depth-wise Convolution

As per the above image, we can clearly see that the total computational cost can be calculated as:

$$D_K \cdot D_K \cdot M \cdot D_F \cdot D_F$$

Point-wise Convolution

Since the depthwise convolution is only used to filter the input channel, it does not combine them to produce new features. So an additional layer called pointwise convolution layer is made, which computes a linear combination of the output of depthwise convolution using a 1×1 convolution.



Point-wise Convolution

As per the image, let's calculate the computational cost again:

$$M \cdot N \cdot D_F \cdot D_F$$

Pointwise convolution cost

So the total computational cost of Depthwise separable convolutions can be calculated as:

$$D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F$$

Depthwise separable convolutions cost

Comparing it with the computational cost of standard convolution, we get a reduction in computation, which can be expressed as:

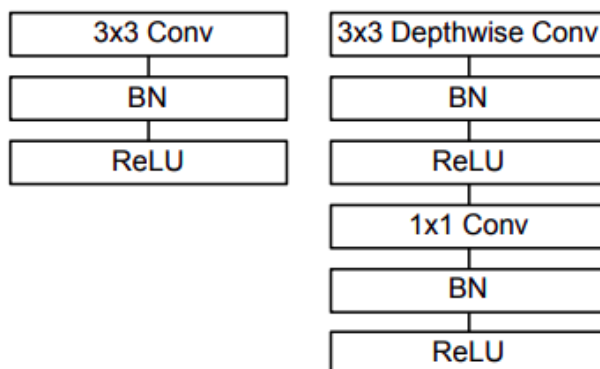
$$\frac{D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F}{D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F} \\ = \frac{1}{N} + \frac{1}{D_K^2}$$

Depthwise separable convolutions cost with standard convolution cost

To put this in a perspective to check the effectiveness of this depthwise separable convolution, let's take an example. Let's take $N=1024$ and $D_K=3$, plugging the values into the equation.

We get 0.112, or in another word, standard convolution has 9 times more the number of multiplication than that of the Depthwise convolution.

2. The entire Network Structure



From the above image, we can see that every convolution layer followed by a batch normalization and a ReLU. Also, a final average pooling is been introduced just before the fully connected layer to reduce the spatial dimension to 1.

Note that the above architecture has 28 layers by counting widthwise and pointwise convolution as separate layers.

Below is architecture table of MobileNet

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5×	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
	Conv dw / s2	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 1024$
	Conv dw / s2	$3 \times 3 \times 1024$ dw
	Conv / s1	$1 \times 1 \times 1024 \times 1024$
	Avg Pool / s1	Pool 7×7
	FC / s1	1024×1000
	Softmax / s1	Classifier

3. Parameters of MobileNet

Although the base MobileNet architecture is already small and computationally not very intensive, it has two different global hyperparameters to effectively reduce the computational cost further. One is the width multiplier and another is the resolution wise multiplier.

Width Multiplier: Thinner Models

For further reduction of computational cost, they introduced a simple parameter called Width Multiplier also refer as α .

For each layer, the width multiplier α will be multiplied with the input and the output channels (N and M) in order to narrow a network.

So the computational cost with width multiplier would become.

$$D_K \cdot D_K \cdot \alpha M \cdot D_F \cdot D_F + \alpha M \cdot \alpha N \cdot D_F \cdot D_F$$

Computational Cost: Depthwise separable convolution with width multiplier

Here α will vary from 0 to 1, with typical values of [1, 0.75, 0.5 and 0.25]. When $\alpha = 1$, called as baseline MobileNet and $\alpha < 1$, called as reduced MobileNet. Width Multiplier has the effect of reducing computational cost by α^2 .

Resolution Multiplier: Reduced Representation

The second parameter to reduce the computational cost effectively. Also known as ρ .

For a given layer, the resolution multiplier ρ will be multiplied with the input feature map. Now we can express the computational cost by applying width multiplier and resolution multiplier as:

$$D_K \cdot D_K \cdot \alpha M \cdot \rho D_F \cdot \rho D_F + \alpha M \cdot \alpha N \cdot \rho D_F \cdot \rho D_F$$

Computational cost by applying width multiplier and resolution multiplier

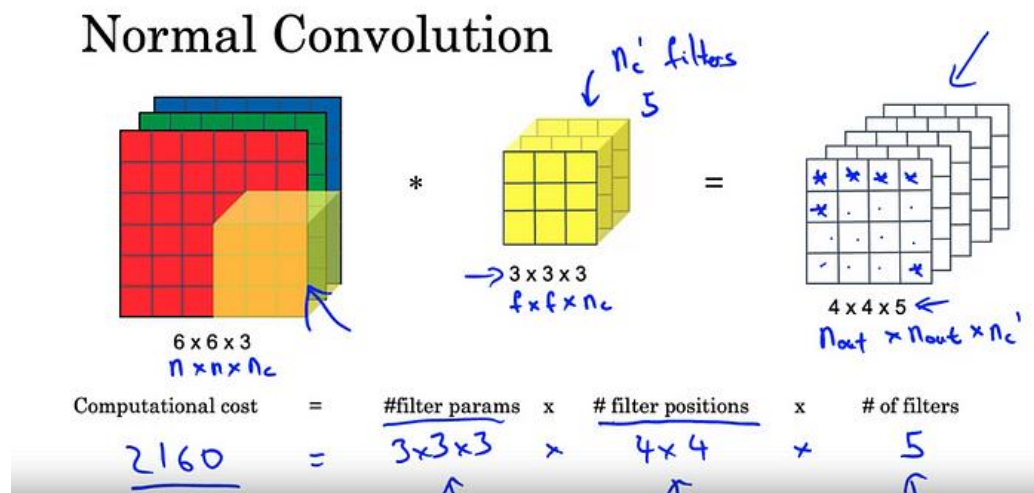
Example:

Normal Convolution and its computational cost:

Let's say we have a $n \times n \times n_c$ input image, where " n_c " is the number of channels and we convolve it using an $f \times f \times n_c$ filter. In normal convolution, you would take this whole volume/filter and perform $f \times f \times n_c$ multiplications, sum them up and you get a single value. We repeat this process for each stride until we get an " $n_{out} \times n_{out} \times n_{out}$ " output. The computational cost would be:

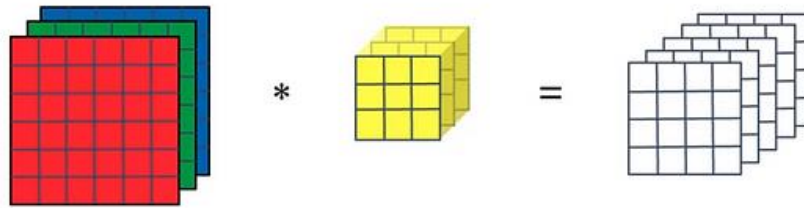
"comp_cost = #filter_params * #filter_positions * #filters"

Normal Convolution

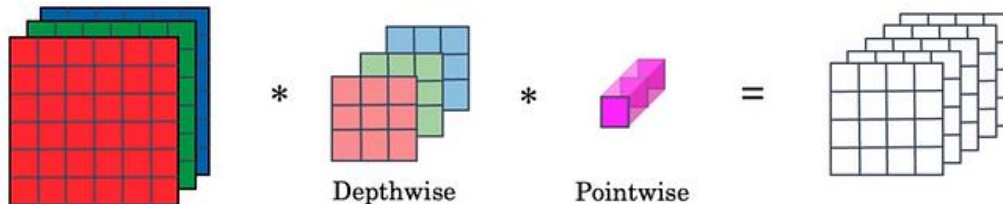


Normal Convolution vs MobileNet:

Normal Convolution



Depthwise Separable Convolution



Depthwise Separable Convolution:

Depthwise separable convolution is the backbone of MobileNet's efficiency. Traditional convolutions perform the spatial convolution on each input channel, resulting in a significant computational burden. However, depthwise separable convolutions split the convolution into two distinct operations: depthwise convolution and pointwise convolution. Now let's see how MobileNet architecture can reduce this cost of 2160 by $\approx 70\%$

Depthwise Convolution:

To illustrate the concept, let's take a compact input image of dimensions 6 x 6 x 3 (width, height, color channels). Considering 3 filters, each with a size of 3 x 3, we observe that each filter convolves through its corresponding color channel, generating an **output of size 4 x 4 x 3**.

It's important to note that the output size is determined by the formula: **output_size = (n — f) + 1**. In our case, with a stride of 1, where n is 6 and f is 3.

The computational cost can be calculated using the formula: "*comp_cost* = #filter_params * #filter_positions * #filters." In this scenario, the **computational cost amounts to 432**, obtained by multiplying the number of filter parameters (3 x 3) with the number of filter positions (4 x 4) and the total number of filters (3).

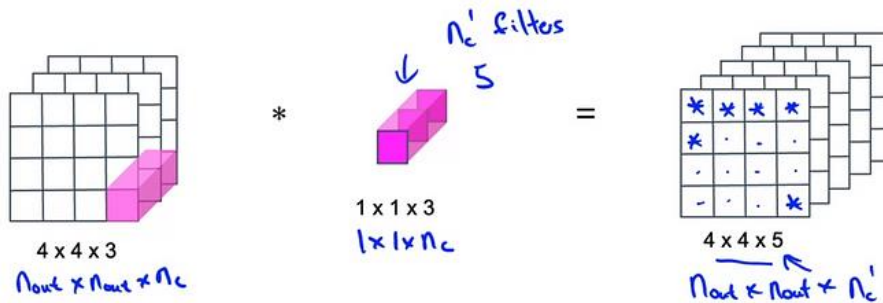
Pointwise Convolution:

After the completion of the depthwise convolution, the pointwise convolution takes place to generate new features by combining the filtered output channels. Pointwise convolutions utilize 1x1 kernels across all input channels, introducing additional non-linearity and enabling the model to grasp intricate patterns.

To execute the pointwise convolution, we utilize the intermediate result obtained from the output of the depthwise convolution, **which, in our case, is a 4 x 4 x 3 matrix denoted as Y**. Subsequently, we apply a 1 x 1 x 3 filter and convolve it across the **matrix Y**. As a result, a 4 x 4

output is produced. If we employ $n = 5$ filters, the final output would be represented by a matrix Z with dimensions of $4 \times 4 \times 5$.

Pointwise Convolution



The computational cost would be:

“comp_cost = #filter_params * #filter_positions * #filters”

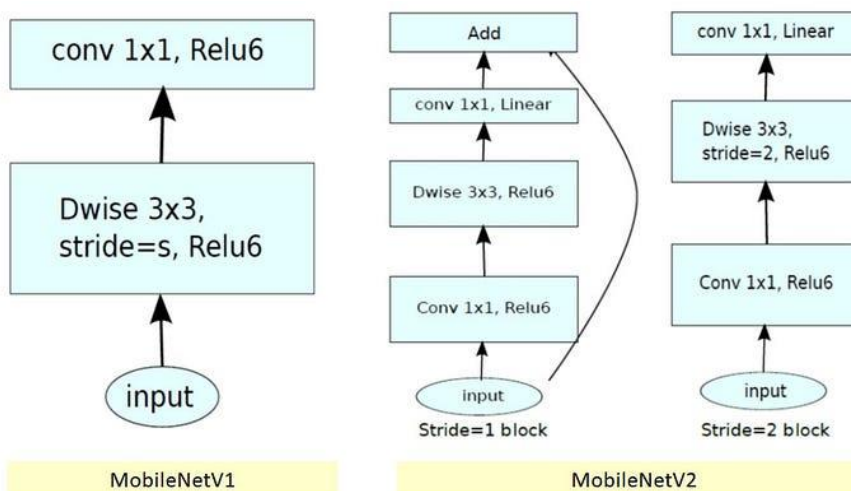
$$240 = (1 \times 1 \times 3) * (4 \times 4) * 5$$

Total Cost Of MobileNet

The total computational cost can be calculated as the sum of the depthwise cost and the pointwise cost, which yields a value of 672. This computation is derived from the addition of the depthwise cost of 432 and the pointwise cost of 240.

Comparatively, the total computational cost for a normal convolution would be 2160, highlighting the significant reduction achieved by utilizing the MobileNet architecture.

MobileNetV2



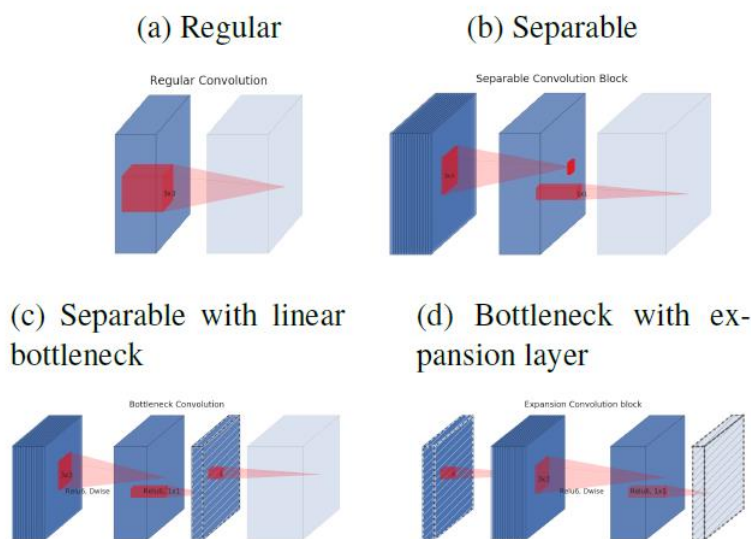
MobileNetV2 pushes the state of the art for mobile-friendly computer vision models by significantly decreasing the number of operations and memory needed, **while retaining the same level of accuracy**.

The authors' main contribution is a **new module**: the **inverted residual with linear bottleneck**. As they put it:

“Our main contribution is a novel layer module: the inverted residual with linear bottleneck. This module takes as an input a low-dimensional compressed representation, which is first expanded to high dimensions and filtered with a lightweight depth-wise convolution. Features are then projected back to a low-dimensional representation with a linear convolution.”

MobileNetV2 builds directly on **MobileNetV1** but introduces new innovations to perform even better in resource-constrained environments, such as mobile devices.

Depth-wise Separable Convolutions:



A key part of MobileNetV2's design is **depthwise separable convolutions**, a technique that was first introduced in **MobileNetV1**. These convolutions are used to reduce the computational load while maintaining accuracy. The convolutions are split into two steps:

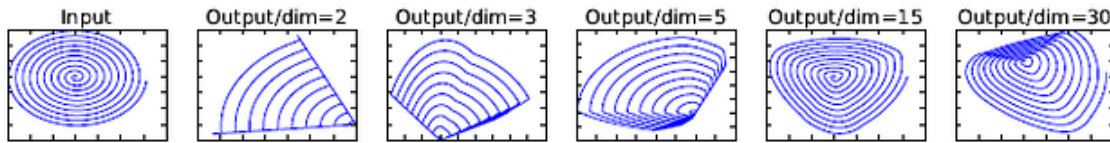
1. **Depthwise Convolution**: Applies a filter to each input channel individually.
2. **Pointwise Convolution**: Uses a 1x1 convolution to combine the outputs from the depthwise step.

As shown in **Figure 1** above, in **regular convolutions** (a), the filter is applied across the entire depth of the input tensor, making the operation computationally expensive. In contrast, **separable convolutions** (b, c, d) break this process into two steps: depthwise convolution (which operates on each input channel separately) and pointwise convolution (which combines the outputs from the depthwise step).

This separable approach reduces the number of parameters and lowers the computational cost by **8 to 9 times** compared to standard convolutions, with only a small reduction in accuracy.

Linear Bottleneck:

One of the paper's major contributions is the **linear bottleneck**. In deep learning, especially in low-dimensional spaces, using non-linear layers like **ReLU** after compressing data can cause important information to be lost. MobileNetV2 addresses this by using **linear layers** instead of ReLU in the bottleneck stages.

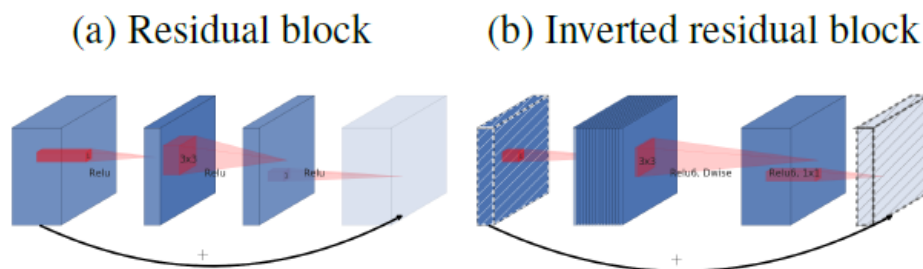


ReLU can distort data in low-dimensional spaces, potentially causing information loss. It shows an input manifold (a spiral) that undergoes **ReLU** transformations when projected into different output dimensions (from **dim=2** to **dim=30**). In low-dimensional spaces like **dim=2** and **dim=3**, you can see that the spiral becomes compressed and distorted, causing certain points of the manifold to collapse together. This leads to significant information loss in the data. However, as the output dimension increases, such as in **dim=15** or **dim=30**, the transformation retains the structure of the manifold much more effectively, meaning there is less information loss.

This highlights why applying **ReLU** in low-dimensional spaces after compression, as seen in standard neural networks, can result in lost features. MobileNetV2's use of **linear bottlenecks** ensures that important features are preserved during the compression and expansion processes, helping the model remain accurate without adding too much computational cost.

Inverted Residuals:

Another key innovation in MobileNetV2 is the **inverted residual block**. Unlike traditional residual blocks (such as those used in ResNet), which expand first and then compress, **inverted residuals** do the opposite. In MobileNetV2, the input is first **compressed** and then **expanded** back to the original dimensions.



As shown in above **Figure**, the inverted residual block (b) uses shortcuts, similar to those used in **ResNet**, but it compresses first and expands second, which makes it more efficient. This “inversion” allows for significant improvements in efficiency by reducing the number of operations required. To maintain performance, **shortcuts** (or **skip connections**) are used between the bottleneck layers, ensuring smooth gradient flow during training and preventing the loss of important information.

Model Architecture:

The **MobileNetV2** architecture is built around **bottleneck residual blocks** (as we discussed earlier), which use **depth-wise separable convolutions** and **linear bottlenecks** to ensure computational efficiency.

One of the most helpful ways to grasp the architecture is through the Table given below, which summarizes how the **bottleneck residual block** transforms input data. It shows how the block takes an input with dimensions $h \times w \times k$, applies a sequence of convolutions, and ultimately outputs a transformed tensor. The table also highlights the use of **ReLU6** as a non-linearity and the **1x1 convolution** used at the end to compress the data back down to the required output dimensions.

Input	Operator	Output
$h \times w \times k$	1x1 conv2d, ReLU6	$h \times w \times (tk)$
$h \times w \times tk$	3x3 dwise s=s, ReLU6	$\frac{h}{s} \times \frac{w}{s} \times (tk)$
$\frac{h}{s} \times \frac{w}{s} \times tk$	linear 1x1 conv2d	$\frac{h}{s} \times \frac{w}{s} \times k'$

The overall structure of **MobileNetV2** is composed of several of these bottleneck layers stacked together. Above Table provides a clear breakdown of how many bottleneck layers are used, the size of the input at each stage, and the operations applied at each layer. It highlights how the architecture starts with a **convolutional layer with 32 filters** and then applies **19 residual bottleneck layers** with different expansion rates and convolutional operations.

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-

In above table each line describes a sequence of 1 or more identical(modulo stride) layers, repeated n times. All layers in the same sequence have the same number c of output channels. The first layer of each sequence has a stride s and all others use stride 1. All spatial convolutions use 3*3 kernels. The expansion factor t is always applied to the input size as described in table.

By examining above table, you can see how the model handles input at different sizes (e.g., 224x224 down to 7x7) and how it gradually reduces the spatial resolution while expanding the number of channels. This structure helps MobileNetV2 balance between *maintaining performance and reducing computational cost*.

In terms of **trade-offs**, the paper mentions hyperparameters like **width multipliers** and **input resolutions**, which can be adjusted to tailor the model to different computational needs. As noted, **MobileNetV2** can scale its performance using width multipliers, allowing it to run on devices with limited resources or deliver higher accuracy for more powerful machines.

To understand how the trade-off between **accuracy** and **computational cost** works in real-world scenarios. Let's say you're deploying **MobileNetV2** on a mobile device with limited processing power. You could reduce the input resolution to **96x96** pixels and apply a **width multiplier** of **0.5**. This would drastically reduce the number of operations and the overall size of the model, making it computationally efficient enough to run on the mobile device. However, this would also come with a slight reduction in accuracy. On the other hand, if you're deploying the model on a more powerful device with greater resources, you can increase the resolution to **224x224** pixels and use a **width multiplier** of **1** or more, which will increase the accuracy at the cost of more computation. This balance between flexibility and performance makes **MobileNetV2** adaptable for a wide range of devices and use cases.

Key Takeaways

Here are the most important takeaways from the paper:

1. MobileNetV2 is designed specifically for **mobile** and **resource-constrained** environments, balancing computational efficiency and accuracy to support real-world applications.
2. By breaking down traditional convolutions into **depthwise** and **pointwise layers**, MobileNetV2 reduces computational cost by 8 to 9 times compared to standard convolutions, while maintaining similar accuracy.
3. A key innovation in MobileNetV2 is the **linear bottleneck** which helps preserves information during compression by removing non-linear activations, which can often lead to loss of important data, especially in low-dimensional spaces.
4. Unlike traditional residual blocks, MobileNetV2 **compresses features before expanding** them again, which allows for better memory efficiency and smoother gradient flow essential for training deeper networks.
5. MobileNetV2, when paired with SSDLite for object detection, proves to be **20 times more computationally efficient** than YOLOv2. It also excels in semantic segmentation tasks when combined with DeepLabv3, performing comparably to larger models like ResNet-101, but with **fewer parameters and lower memory requirements**.
6. MobileNetV2's architecture allows users to adjust the **network width, resolution, and multipliers**, enabling it to scale across different devices and tasks. This flexibility makes it an ideal choice for diverse applications, from mobile devices to embedded systems.